Запитання для самоперевірки з усіх лекцій

lecture_01_a.md (Основи UX-дизайну: Принципи та патерни поведінки користувачів)

1. Що таке UX-дизайн і чим він концептуально відрізняється від UI-дизайну?
2. Назвіть 3 ключові характеристики "зручного" (usable) веб-інтерфейсу. Чому швидкість завантаження також є частиною UX?
3. Поясніть концепцію "Ментальної моделі" користувача. Чому використання стандартної іконки "лупи" для пошуку краще, ніж вигадування власного креативного символу?
4. Що таке "патерн F-подібного читання" (F-pattern) і як він впливає на розташування ключового контенту на сторінці?
5. Опишіть правило "3 кліків". Чи є воно догмою в сучасному вебі? Що важливіше за мінімальну кількість кліків по навігації?

lecture_01_b.md (Дослідження та тестування користувацького досвіду)

1. Хто такі "User Personas" (Персонажі) і яку практичну користь вони несуть при розробці структури сайту?
2. В чому різниця між кількісними (Google Analytics) та якісними (Інтерв'ю) дослідженнями аудиторії?
3. Що таке "Customer Journey Map" (CJM)? З яких етапів зазвичай складається шлях клієнта в e-commerce додатку?
4. Поясніть алгоритм проведення A/B тестування. Скільки елементів можна змінювати на сторінці одночасно для отримання достовірних результатів B-версії?
5. Що таке "Коридорне тестування" (Hallway testing) і які критичні та неочевидні помилки інтерфейсу воно дозволяє виявити на ранніх етапах?

lecture_01_c.md (Основи UI-дизайну та візуального оформлення)

1. Що таке "Теорія кольору" в UI? Як пов'язані між собою поняття "Primary color", "Secondary color" та "Accent color" у дизайн-системі?
2. Яка різниця між шрифтами сімейства Serif (із зарубками) та Sans-serif (без зарубок)? Які з них рекомендується використовувати для великих масивів тексту на екранах моніторів?

3. Що таке "Ієрархія типографіки" і які CSS параметри (розмір, товщина, колір) допомагають людині швидко відсканувати сторінку, ще не вчитуючись у неї?
4. Поясніть вплив контрасту кольорів на доступність інтерфейсу (a11y). Який мінімальний коефіцієнт контрастності для тексту за стандартом WCAG 2.1 AA?
5. Що таке мікроінтеракції (micro-interactions) і як візуальний фідбек (наприклад, зміна кольору кнопки при наведенні або спіннер завантаження) покращує досвід користувача?

lecture_01_d.md (Побудова інтерфейсів: Сітки та інтерактивні елементи)

1. Що таке "Модульна сітка" (Grid system) в дизайні інтерфейсів? Чому класична сітка Bootstrap/Figma найчастіше базується саме на 12 колонках?
2. Поясніть принцип "Теорії близькості" (Law of Proximity) з Гестальт-психології. Як відступи (margins/paddings) впливають на логічне групування елементів (наприклад, заголовка, тексту та кнопки карток)?
3. В чому різниця між "Whitespace" (макро відступи між блоками сайту) і мікро-відступами? Чому "пустота" (повітря) на сайті — це теж важливий елемент дизайну, якого не варто боятися?
4. Опишіть три стани (states) обов'язкової промальовки інтерактивної форми вводу (`input`): Default, Focused та Error. Яке візуальне кодування найкраще підходить для Error state?
5. Що таке "Закон Фітца" (Fitts's Law) при розрахунку розміру клікабельних елементів (кнопок) і їх розташування на екрані, особливо на мобільних телефонах?

lecture_01.md

1. В чому різниця між поняттями Інтернет (Internet) та Всесвітня Павутина (WWW)?
2. Чому протокол TCP/IP називають "надійним", а UDP — "швидким"? Де саме використовуються ці два підходи?
3. Опишіть процес розпізнавання доменного імені за допомогою DNS (від кешу браузера до Root-серверів).
4. Яка проблема протоколу HTTP/1.1 була вирішена за рахунок впровадження мультиплексування в HTTP/2? Які переваги дає HTTP/3 порівняно з TCP-протоколами?
5. Поясніть відмінність між HTTP-методами `POST` та `PUT` в контексті дій.
6. Що означають групи статус-кодів 2xx, 3xx, 4xx і 5xx? Яка ключова різниця між 401 та 403?
7. Чим ідея "Semantic Web (Web 3.0)" (базується на розумінні змісту через мікродані та AI) відрізняється від парадигми "децентралізованого Web3" (сучасний контекст блокчейну)?

lecture_02.md

1. Що таке теги в HTML? В чому різниця між тегом та елементом?

2. Яким є головне призначення мета-тегу `<meta name="viewport"...>` (у розділі `<head>`) і що трапиться з мобільним відображенням додання цього тегу?
3. В чому принципова відмінність "Блокових" (block) і "Рядкових" (inline) елементів? Наведіть приклади.
4. Поясніть на прикладах, у яких випадках використовують семантичні теги `<article>` замість `<section>`, і де саме логічно розміщувати `<aside>`?
5. Для чого призначений атрибут `alt` у тегу `<img>`? Які дві проблеми він допомагає вирішити? Назвіть вийняток, у якому `alt` можна лишити зовсім пустим (`alt=""`).
6. Що таке Web Accessibility (a11y) і як працюють інструменти "skrinreder'и" (екранні читалки)?
7. Яка роль атрибута `tabindex` і чому використання `tabindex="1"` чи інших позитивних чисел вважається поганою практикою (анти-патерном) в UI/UX дизайні?
8. Поясніть роботу і застосування атрибутів `aria-hidden` та `aria-label` для кастомних JS віджетів (наприклад, звичайних кнопок-іконок).

lecture_03.md

1. Які чотири складові містить блокова модель CSS?
2. Що змінює правило `box-sizing: border-box` у розрахунку розмірів елемента?
3. В чому різниця між властивостями `display: none` та `visibility: hidden`?
4. Які стилі застосовуватимуться до елемента, якщо він є `display: inline` і йому задати `width: 100px` та `margin-top: 50px`?
5. Що таке "схлопування зовнішніх відступів" (margin collapsing)?
6. Відносно чого позиціюється елемент із `position: absolute`? Що станеться, якщо у нього немає жодного спозиційованого батьківського елемента?
7. Чим `position: sticky` відрізняється від `position: fixed`?
8. Чому елемент із `z-index: 999` може знаходитись ПІД елементом із `z-index: 2`? У яких випадках це відбувається?

lecture_04.md

1. У чому принципова відмінність макетування на базі Flexbox від підходів із використанням `float`?
2. Яку властивість необхідно додати контейнеру (батьку), щоб розпочати роботу з Flexbox?
3. Де знаходиться Головна (Main) і Поперечна (Cross) осі, якщо задано `flex-direction: column`?
4. Яка властивість контейнера відповідає за обгортання елементів на новий рядок, якщо вони не вміщуються по ширині?

5. Як центрирувати елемент всередині Flex-контейнера одночасно по горизонталі та вертикалі?
6. Чим `justify-content` відрізняється від `align-items`?
7. Що означають три значення у властивості-скороченні `flex: 1 0 auto`?
8. Як впливає властивість `order` на позиціонування елементів? З якого значення починається нумерація?

lecture_05.md

1. У яких ситуаціях варто використовувати Flexbox, а у яких — CSS Grid?
2. Що таке лінія сітки (Grid Line) і з якої цифри починається їх нумерація?
3. Як працює одиниця вимірювання `fr` (Fraction)? Наведіть приклад.
4. Поясніть принципову різницю між `auto-fill` та `auto-fit`.
5. Як властивість `span` впливає на розташування елемента по сітці?
6. Що таке іменовані області (`grid-template-areas`) і як з ними працювати? Наведіть приклад синтаксу для 2-колонкового макета з `header` і `footer`.
7. Який мета-тег є обов'язковим для коректної роботи адаптивного дизайну на мобільних пристроях?
8. Поясніть концепцію Mobile First. Які медіа-запити (`min-width` чи `max-width`) використовуються у цьому підході?

lecture_06.md

1. Для чого призначений атрибут `action` у тегу `<form>`?
2. У чому семантична та практична різниця між HTTP-методами `GET` та `POST` при відправці форми?
3. Чим відрізняється поведінка групи елементів `type="radio"` від групи елементів `type="checkbox"`? Завдяки якому атрибуту групуються радіо-кнопки?
4. Опишіть призначення атрибута `name` на елементі вводу. Що станеться, якщо його пропустити?
5. Наведіть щонайменше 2 переваги використання нових типів вводу HTML5 (наприклад, `type="email"`, `type="date"`).
6. Чому важливо використовувати тег `<label>`? Як саме його "пов'язують" із текстовим полем?
7. У чому різниця між атрибутами стану `disabled` та `readonly`?
8. Що таке клієнтська валідація? Чому, якщо вона є, все одно необхідна валідація даних на бекенді (сервері)?
9. Який атрибут у HTML5 застосовується для перевірки значення поля за допомогою регулярного виразу (regex)?

lecture_07.md

1. Чим растрова графіка онцептуально відрізняється від векторної (SVG)? Де варто використовувати кожну з них?
2. Які переваги пропонують формати WebP або AVIF порівняно із JPEG і PNG?
3. Що таке "Ліниве завантаження" (Lazy Loading) і як його реалізувати стандартними засобами HTML? Чому його не можна застосовувати для всіх зображень на сайті?
4. Навіщо потрібно вказувати атрибути `width` та `height` для тегу `<img>`? Як це впливає на метрику CLS (стабільність макета)?
5. Що таке атрибут `srcset` і яка його головна роль (для мобільних телефонів vs великих дисплеїв)?
6. Які проблеми вирішує елемент `<picture>` з дочірніми `<source>` (назвіть мінімум дві причини використання)?
7. Що означає термін "ресурс, блокуючий рендерінг" і чому CSS вважається саме таким ресурсом? Як мініфікація допомагає вирішити цю проблему?

lecture_08.md

1. Поясніть своїми словами взаємозв'язок між Доменом, DNS та Хостингом.
2. Чому розгортання сайту без SSL-сертифіката (по HTTPS) є неприпустимим у сучасному вебі?
3. Що таке механізм Continuous Deployment (CD) в контексті платформ на кшталт Vercel чи Netlify?
4. У чому різниця між етапами Crawling (Сканування) та Indexing (Індексування) у роботі пошуковика Google?
5. Які типи HTML-елементів (тегів та атрибутів) є найбільш критичними для внутрішнього (технічного) SEO? Чому на сторінці має бути лише один тег `<h1>`?
6. Ви відправили посилання на свій сайт другу в Telegram, але замість гарної картинки він отримав просто голий текст і адресу. Чого не вистачає у вашому коді?
7. Що таке Lighthouse і які чотири основні метрики він аналізує?

lecture_09.md

1. В чому полягає критична проблема використання `var` для створення змінних, і коли замість `const` слід використовувати `let`?
2. Назвіть 5 примітивних типів даних у JS. У чому їхня глобальна відмінність від Об'єктів?
3. Що буде результатом виразу `typeof null` згідно з документацією (це відомий баг мови)? Що таке `undefined`?
4. Навіщо у HTML і JS придумали два оператори порівняння: подвійне (`==`) і потрійне (`===`) дорівнює?

5. З якого числа прийнято починати нумерацію елементів будь-якого масиву? Як отримати останній елемент масиву, якщо його довжина заздалегідь невідома?
6. Що таке об'єкт в JS? У якому форматі там зберігаються дані?
7. Створіть змінну `myString` використовуючи шаблонні рядки, яка міститиме текст "Разом: X грн", де X береться із заданої числової змінної `totalPrice`.
8. Опишіть своїми словами, що робить оператор `...` (Spread) і як його застосувати для злиття двох масивів у третій?

lecture_10.md

1. Чим відрізняється Function Declaration від Function Expression?
2. В чому специфіка стрілочних функцій (Arrow Functions) окрім їхнього короткого синтаксису? Чого вони "НЕ мають"?
3. В чому різниця між Rest оператором та Spread оператором, коли обидва виглядають як `...`?
4. Опишіть своїми словами, що таке Замикання (Closure). Для якої найпопулярнішої проблеми його використовують?
5. Що виведе `console.log(this)` в глобальній області видимості в браузері? А всередині методу об'єкта?
6. Якщо у вас є метод об'єкта `user.showName()`, і ви передасте його як callback (наприклад у таймер: `setTimeout(user.showName, 1000)`), швидше за все контекст загубиться. Як вижити в цій ситуації за допомогою методів `bind` або стрілочної функції?
7. У чому принципова різниця між методами функції `call()` та `bind()`?

lecture_11.md

1. З якими аргументами викликається Callback-функція в методах масиву? Який з них є обов'язковим для використання?
2. В чому принципова різниця між `forEach` та `map`? У яких випадках слід обрати кожен із них?
3. Що буде, якщо у Callback-функції для методу `map` ви забудете написати команду `return`? Яким буде результат (масив з чим)?
4. Метод `filter` отримав Callback, який завжди повертає число `1`. Що буде у відфільтрованому масиві?
5. Для чого використовується значення `accumulator` у методі `reduce`, і де в синтаксисі вказується початкове значення цього акумулятора?
6. Якщо вам потрібно знайти користувача "Ігор" і пересвідчитися, що він є в базі, чому метод `find` буде набагато ефективнішим вибором (з точки зору швидкодії сервера), ніж метод `filter`?

7. У вас є масив чисел `[5, 1, 15]`. Яким буде результат виклику `arr.sort()` без переданої Callback-функції і чому?
8. Що таке Chaining (ланцюжок методів) і завдяки якій властивості методів `filter` або `map` він став можливим?

lecture_12.md

1. Що таке DOM-дерево і чим воно принципово відрізняється від статичного HTML-файлу?
2. Який метод потрібно використати, якщо ви хочете знайти всі кнопки з класом `.btn` на сторінці? Який тип даних він поверне?
3. В чому небезпека використання властивості `innerHTML` для вставки даних, які ввів користувач? Яка властивість безпечніша?
4. Чому для зміни зовнішнього вигляду елемента за допомогою JS рекомендується використовувати методи конструкції `classList`, а не пряму зміну властивості `style`?
5. Напишіть код, який створить новий тег `<li>` з текстом "Яблуко" і додасть його в кінець списку `<ul>` з класом `.fruits`.
6. Опишіть своїми словами, що таке "спливання подій" (Bubbling). Чому об'єкт події (`event`) такий важливий у патерні "Делегування"?
7. Уявіть, що JS виконує складний математичний цикл `for` на мільярд ітерацій (синхронний код). Що станеться зі сторінкою? Чому?
8. Що таке Call Stack (Стек викликів) та Task Queue (Черга завдань), і яку роль між ними відіграє Event Loop?

lecture_13.md

1. Чим відрізняється HTTP Метод `GET` від методу `POST` у концепції REST API? Який з них викликається функцією `fetch()` за замовчуванням?
2. Вкажіть три правила (відмінності), які відрізняють формат тексту JSON від класичного об'єкта JavaScript у пам'яті?
3. Що таке Promise в JavaScript, і в яких трьох станах він може перебувати?
4. Яка роль ключового слова `await`, чому без нього функція `loadData()` видала б помилку, або просто повернула Promise `Pending`? Де саме його потрібно ставити?
5. Напишіть рядок коду (лише рядок), який перетворить отриманий по HTTP текстовий JSON із масивом у реальний масив даних JS.
6. Навіщо у конфігурації (в `options`) при здійсненні `POST`-запиту ми зобов'язані передати параметр headers `Content-Type: application/json`?
7. Чому функція `fetch` може працювати без генерації системної помилки ігноруючи навіть статус "404 Not Found"? Як з цим боротися?

8. Яке призначення блоку `try...catch` при написанні функції з мережевими запитами? Що конкретно виконує секція `catch(error)`?

lecture_14.md

1. Чому розробка великих веб-додатків тільки з використанням Vanilla JS (та інструментів на зразок `document.querySelector`) є поганою практикою сьогодні?
2. Поясніть своїми словами ключову різницю між фреймворком (наприклад, Angular) та бібліотекою (наприклад, React).
3. Що таке механіка SPA (Single Page Application)? Як вона впливає на користувацький досвід та споживання трафіку порівняно зі старими багатосторінковими сайтами?
4. Які три головні переваги дає розбиття великого сайту на маленькі незалежні компоненти?
5. Наведіть приклад імперативного та декларативного підходу до вирішення побутової проблеми за межами програмування (на кшталт поїздки на таксі). Як це стосується React?
6. Що таке Virtual DOM? Чому React не вносить зміни безпосередньо у справжній DOM браузера одразу після кліку користувача?
7. Поясніть крок за кроком, що робить алгоритм Реконсиляції (Reconciliation), коли у додатку змінюються дані.
8. Чому індустрія масово відмовилася від `Create React App` на користь інструментарію `Vite`?

lecture_15.md

1. Чому всі React-компоненти обов'язково повинні називатися з великої літери?
2. Для чого існують React Фрагменти (`<> </>`)?
3. Дайте визначення терміну `Props`. У якому напрямку між компонентами можлива їхня передача? Чи має право син-компонент поміняти те, що йому передали?
4. Напишіть приклад коду, як за допомогою деструктуризації витягнути з пропсів параметри `title` та `color` (де `color` за замовчуванням буде "black").
5. Чому React змушує нас використовувати атрибут `className` замість класичного HTML-атрибута `class`?
6. За допомогою якого методу масиву здійснюється масове створення (рендерінг) компонентів у вигляді списку?
7. Чому використання випадкових чисел (на кшталт `Math.random()`) або індексів масиву у якості `key` в списках — це погана практика і може викликати страшні баги? Яке джерело для значення атрибута `key` є еталонним?
8. Що таке умовний рендеринг і які два оператори JavaScript використовуються для відображення або приховування елементів у JSX на його основі?

lecture_16.md

1. Чим `State` фундаментально відрізняється від `Props` з точки зору права на зміну даних?
2. Що станеться, якщо ви спробуєте оновити значення лічильника напряму через математичний оператор (наприклад, `count = count + 1`), не використовуючи функцію `setCount()`?
3. Чому виклики хуків (усіх функцій зі словом `use`) заборонено ставити всередину умовних конструкцій `if () {...}`?
4. Що означає термін "асинхронне оновлення стану"? Яке значення побачить console.log(X) одразу після рядка з виконання `setX(...)`?
5. Чому React "категорично забороняє" використовувати вбудований метод `push()` для масивів, які зберігаються у стейті? Як правильно додати елемент?
6. Що таке Керований Компонент (Controlled Component) на прикладі поля вводу `<input />`? Які два атрибути (пропси) обов'язкові для його правильної роботи?
7. Поясніть логіку "Підняття стану". Якщо брат і сестра компоненти потребують одних даних, де варто оголосити змінну через `useState`?

lecture_17.md

1. Згадайте визначення "Чистої функції". Чому запит на сервер (`fetch`) або запуск таймера `setInterval` вважаються Побічними Ефектами, що порушують чистоту?
2. Які три основні фази має життєвий цикл кожного окремого компонента в інтерфейсі?
3. Що буде, якщо ми взагалі не передамо другий аргумент (масив) до функції `useEffect`? Коли саме він запуститься?
4. Для чого в 90% випадків ідеально підходить хук `useEffect` із порожнім масивом залежностей `[]`? Як часто він виконується?
5. Чому React видаватиме попередження в консолі, якщо ви спробуєте написати `useEffect(async () => {...}, [])`? Як правильно робити асинхронні виклики всередині цього хука?
6. Наведіть сценарій, коли функція "Очищення" (та, яку ефект повертає через `return () => {}`) є абсолютно необхідною для запобігання витоку пам'яті? Коли саме React її викличе?
7. Поясніть покроково, як відсутність порожнього масиву `[]` при виклику `fetch` (який закінчується командою `setData`) призводить до безкінечного циклу зависання браузера?

lecture_18.md

1. Чому React Router є сторонньою бібліотекою, а не вбудований у React? Що таке клієнтська маршрутизація?
2. Поясніть шкоду, яку спричинить тег `<a>` замість `<Link>` для навігації в межах SPA-додатку.
3. Що означає двокрапка `:` у рядку `/product/:id` під час налаштування маршруту? Який вбудований хук React Router допоможе компоненту витягти значення на місці цього двокрапки з адреси браузера?
4. В якій ситуації і для якої поведінки вам би довелося застосувати хук `useNavigate()` замість `<Link>`?
5. Для чого на сторінці-шаблоні Батьківського Маршруту (Parent Layout) потрібно вставляти компонент `<Outlet />`? Під час розробки яких поширених UI-шаблонів це корисно?
6. Як налаштувати в React Router обробку невідомих маршрутів, щоб створити сторінку "404 Not Found" (Який "символ" відповідає за перехоплення всіх неспівпадінь)?

lecture_19.md

1. Змоделюйте і опишіть ситуацію, яка ілюструє жах архітектурного паттерну під назвою `Prop Drilling`.
2. Які дані має сенс залишати локальними (через звичайний хук `useState`), а які доцільно винести на Глобальний рівень?
3. Які два ключові кроки (компоненти/хуки) потрібно реалізувати розробнику, щоб створити свій перший Context API та доставити в нього інформацію?
4. Назвіть функції компонента `<ThemeContext.Provider>`, навіщо йому атрибут `value`?
5. У чому полягає критична слабкість Context API під час керування великим об'єктом стану, який містить 50 різних полів і щомиті оновлюється?
6. Назвіть 2 головні відмінності і переваги бібліотеки `Zustand` над класичним рішенням з Context API, коли мова заходить про швидке розгортання проєкту.

lecture_20.md

1. У чому полягає найбільша небезпека (і біль) використання одного глобального файлу `style.css` у великих веб-додатках (до епохи ізоляції)? Що таке "Специфічність CSS"?
2. Поясніть принцип "CSS-in-JS" (на прикладі Styled Components). Що фізично імпортується і що ми пишемо у зворотному апострофі?
3. В чому головна супер-сила Styled components порівняно навіть з CSS Modules? Що компонент CSS-in JS може "спіймати" від своїх батьків для перетворення кольору/ширини динамічно?
4. Розшифруйте ідеологію "Utility-First" на якій побудовий Tailwind CSS? Як би ви описали клас `.card` і `.btn` у термінах мікро-класів (просторо/колір)?

5. Що означають класи Tailwind `m-4`, `p-2`, `bg-blue-600`, `hover:text-red-300`, `flex`?
6. З яким головним "Мінусом" Tailwind стикаються більшість розробників, коли переходять на нього (чисто візуальна естетика вашого коду)?
7. Яким чином Tailwind гарантує, що незважаючи на тисячі доступних класів-утиліт, фінальний CSS-файл для браузера буде "важити" лічені кілобайти?

lecture_21.md

1. Чому Node.js став революцією і звідки він взяв свій головний "двигун" (Engine) для обробки коду?
2. Поясніть принцип "неблокуючого вводу/виводу" (Non-blocking I/O). Що робить сервер на Node.js, поки чекає інформацію від повільної бази даних?
3. Що таке `npm` і яку роль цей концепт відіграє в екосистемі Node?
4. Для чого Frontend-розробники підключають фреймворк `Express` замість того, щоб масувати обробку запитів голим Node.js модулем "http"?
5. Що таке Middleware (Мідлвари) у філософії Express? Навіщо вони потрібні та яку функцію обов'язково треба викликати всередині мідлвару, щоб код працював далі (згадайте функцію під кодом `next`)?
6. Ви отримали клієнтський `GET` запит вигляду `/users/99`. Як називається об'єкт (його поле), всередині якого лежить число 99, і як з ним працює Express-навігація (`req.xxx...`)?
7. Ваш сервер має прочитати повідомлення з форми (POST запит) від користувача і записати у таблицю бази. Який вбудований мідлвар треба додати у верхній частині файлу конфігурації `app.js`, щоб об'єкт `req.body` не був `undefined`?

lecture_22.md

1. Чим фундаментально відрізняється таблиця SQL (наприклад PostgreSQL) від колекції NoSQL (наприклад Firestore)? Яка з цих архітектур менш гнучка до зміни кількості характеристик сутності на льоту?
2. Що таке BaaS? Як цей підхід рятує життя Frontend-інженерам чи мобільним розробникам-одинакам під час створення власних проектів та стартапів?
3. В чому головна архітектурна різниця "під капотом" між конкурентами Firebase та Supabase? Яка база є реляційною?
4. Які змінні конфігурації (які зазвичай отримують в адмін-панелі) обов'язково треба передати функції `createClient()` для коннекту клієнта з хмарою, і де прийнято їх приховувати в проекті?
5. Напишіть за допомогою чейнінгу методів `supabase`, як зібрати і відсортувати список з 10 товарів.

6. Як працює Аутентифікація в Supabase: які дві базові функції викликаються для реєстрації юзера та для його логіну? Куди платформа непомітно зберігає "перепустку" (токен) в межах браузера?
7. Сценарій: Хакери можуть прочитати ваші ключі Supabase у відкомпільованому `bundle.js` сайті і спробувати викликати з консолі метод `supabase.delete()`, щоб обнулити вам базу. Який механізм інженерного захисту створений для блокування цього?

lecture_23.md

1. Згадайте Піраміду тестування. Який відсоток тестів мають займати Unit та E2E тести, і чому E2E не роблять на кожну маленьку функцію?
2. Як називаються три ключові кроки будь-якого правильного тесту (Підготовка, Дія, Перевірка) англійською мовою (правило трьох 'A')?
3. У коді Jest ми бачимо `expect(someVar).toBe(5)`. Який аналог цієї функції буде використано для порівняння двох складних об'єктів з ключами (`toEqual`)?
4. В чому полягає філософія React Testing Library (RTL)? Чому він шукає елементи на екрані за їхнім текстом `getByText`, а не за ім'ям CSS класу (наприклад `.login-button`)?
5. Назвіть 3 базові команди Cypress для взаємодії з веб-сторінкою: відкрити URL, знайти елемент і натиснути його, набрати текст у `input`.
6. Розшифруйте абревіатуру TDD та поясніть 3 фази циклу "Червоний-Зелений-Рефакторинг".

lecture_24.md

1. Згадайте, що таке JavaScript помилка "Runtime Exception". В який момент часу і де саме вона "вибухає" для звичайного Юзера, коли ми просто використовуємо ванільний JS? А коли TS зупиняє цю помилку?
2. Браузер (Chrome чи Safari) має вбудований безкоштовний компілятор TypeScript, щоб читати цей новий синтаксис? Що фізично потрапить у кінцевий файл `bundle.js`, який віддає сервер на хостингу?
3. Що у філософії TypeScript означає знак питання `?` відразу після імені ключа об'єкта (в межах "Interface")? Яке значення JS може ховатися на цьому місці окрім бажаного стрінгу (рядка)?
4. Поясніть концепцію та призначення оператора вертикальної лінії `|` під назвою "Union type". Коли це може знадобитися у роботі з функцією приймання ідентифікаторів товарів з бази даних?
5. Для чого ставиться ключове слово `void` після закриваючої круглої дужки функції `function sayHello(...) : void`?
6. Якщо ваш компонент `UserCard` приймає в себе пропси імені, та статусу (active чи не active), як за допомогою TypeScript захистити іншого розробника від того, щоб він випадково не

передав цей `status` у форматі звичайного стрінгу "OK"? Складіть цей "Контракт Пропсів" в голові.